

Universidad Nacional Autónoma de México
Facultad de Ingeniería



Práctica No.06
Algoritmo de Cifrado Vernam

Asignatura:

CRIPTOGRAFÍA

Profesor: *Dr. Alfonso Francisco De Abiega L'Eglisse*

Grupo: 08

Integrantes:

Espinoza Matamoros Percival Ulises
García Cortés Adolfo de Jesús
Lugo Manzano Rodrigo
Montiel Juárez Oscar Iván

Semestre: 2027-1

Práctica 06. Algoritmo de Cifrado: Vernam

1. Explicación del Método

1.1. Letras como bits

Cada letra se cambia por su número (`a`=0, ..., `z`=25) y ese número se escribe con **5 bits**. Por ejemplo, la `h` es el 7, que en binario es `00111`.

1.2. La operación XOR

El XOR compara dos bits: si son iguales da 0 y si son distintos da 1.

a	b	$a \oplus b$
0	0	0
0	1	1
1	0	1
1	1	0

1.3. Cifrado y descifrado

La clave K debe tener la misma longitud que el mensaje M (una letra de clave por cada letra del mensaje). El cifrado y el descifrado son la misma operación:

$$C = M \oplus K \qquad\qquad M = C \oplus K$$

Esto funciona porque $K \oplus K = 0$, así que al hacer XOR otra vez con la clave se recupera el mensaje.

Ejemplo: la letra `h` (`00111`) con la letra de clave `k` (`01010`):

$$00111 \oplus 01010 = 01101\ (13)\ \rightarrow\ \texttt{n}$$

1.4. Un detalle importante

Con 5 bits se pueden escribir los números del 0 al 31, pero el alfabeto solo llega hasta el 25. Por eso, el resultado del XOR puede ser un número entre 26 y 31, que **no corresponde a ninguna letra**. Por esta razón el programa guarda el criptograma como una lista de grupos de 5 bits y no como texto.

2. Implementación en Python

2.1. Funciones de apoyo

```python
def a_bits(numero):
    #Convierte un número (0-31) a una cadena de 5 bits.
    return format(numero, "05b")


def xor_bits(bits1, bits2):
    #XOR bit a bit: bits iguales -> 0, bits distintos -> 1.
    resultado = ""
    for b1, b2 in zip(bits1, bits2):
        resultado += "0" if b1 == b2 else "1"
    return resultado


def generar_clave(largo):
    #Genera una clave aleatoria de letras con la misma longitud que el mensaje.
    return "".join(random.choice(A) for _ in range(largo))
```

- `a_bits` pasa un número a una cadena de 5 bits, con ceros a la izquierda.
- `xor_bits` compara los dos grupos de bits uno por uno.
- `generar_clave` crea una clave aleatoria del mismo largo que el mensaje.

2.2. Funciones de cifrado y descifrado

```python
def cifrar(mensaje, clave, mostrar_detalle=False):

    #Vernam: C = M xor K (bit a bit, 5 bits por letra).
    #Retorna una lista de cadenas de 5 bits, porque el resultado puede ser
    #un valor entre 26 y 31 que ya no corresponde a una letra.

    texto = mensaje.replace(" ", "")
    criptograma = []

    for i, caracter in enumerate(texto):
        bits_m = a_bits(A.index(caracter))
        bits_k = a_bits(A.index(clave[i]))
        bits_c = xor_bits(bits_m, bits_k)
        decimal = int(bits_c, 2)

        if mostrar_detalle:
            nota = f"'{A[decimal]}'" if decimal < 26 else "(no es letra)"
            print(f"'{caracter}' {bits_m} xor '{clave[i]}' {bits_k} = {bits_c} ({decimal:2d}) {nota}")

        criptograma.append(bits_c)

    return criptograma


def descifrar(criptograma, clave, mostrar_detalle=False):

```

```
    # M = C xor K (porque K xor K = 0).

    mensaje = ""

    for i, bits_c in enumerate(criptograma):
        bits_k = a_bits(A.index(clave[i]))
        bits_m = xor_bits(bits_c, bits_k)
        decimal = int(bits_m, 2)
        letra = A[decimal]

        if mostrar_detalle:
            print(f"{bits_c} xor '{clave[i]}' {bits_k} = {bits_m} ({
                decimal:2d}) -> '{letra}'")

        mensaje += letra

    return mensaje
```

La función de cifrado regresa una *lista de cadenas de 5 bits*, porque el resultado puede valer entre 26 y 31 y ya no ser una letra. Cuando el valor sí es letra, el detalle lo muestra; si no, escribe `(no es letra)`. La función de descifrado hace el XOR con la misma clave y siempre da un número entre 0 y 25, así que sí se puede convertir a letra.

2.3. Validación de datos

```
# Validación de la clave (|K| = |M|; si es más larga se usan las primeras
    letras)
while True:
    clave = input(f"> Ingrese la clave de al menos {total_letras} letras (
        Enter para generarla aleatoria): ").lower()

    if clave == "":
        clave = generar_clave(total_letras)
        print(f"  Clave generada: {clave}")
        break

    es_valida = True
    for caracter in clave:
        if caracter not in A:
            es_valida = False
            break

    if es_valida and len(clave) >= total_letras:
        clave = clave[:total_letras]
        break
    else:
        print(f"\n![Error]: La clave debe tener solo letras minúsculas (a-
            z) y al menos {total_letras} letras.\n")
```

El mensaje se trabaja sin espacios. La clave debe tener solo letras minúsculas y al menos tantas letras como el mensaje; si es más larga, se usan las primeras. Si el usuario solo presiona Enter, el programa genera una clave aleatoria.

3. Pruebas del Programa

3.1. Prueba 1: mensaje “hola equipo diez”

El mensaje tiene 14 letras. Se usó una clave de 14 letras (`keyvaluesecret`, que sale de recortar `keyvaluesecretx`). En la Figura 1 se ve que dos letras (`q` y `p`) dieron valores 27 y 29 al cifrarse, que no son letras.

```
PS C:\Users\adolf\AppData\Local\Programs\Microsoft VS Code> & 'C:\Users\adolf\AppData\Local\Programs\Python\Python311\python.exe' '"c:/Users/a

  Algoritmo de Cifrado Vernam

> Ingrese el mensaje a cifrar: hola equipo diez
> Ingrese la clave de al menos 14 letras (Enter para generarla aleatoria): keyvaluesecret

--- Cálculo del cifrado (fórmula C = M xor K) ---
'h' 00111 xor 'k' 01010 = 01101 (13) 'n'
'o' 01110 xor 'e' 00100 = 01010 (10) 'k'
'l' 01011 xor 'y' 11000 = 10011 (19) 't'
'a' 00000 xor 'v' 10101 = 10101 (21) 'v'
'e' 00100 xor 'a' 00000 = 00100 ( 4) 'e'
'q' 10000 xor 'l' 01011 = 11011 (27) (no es letra)
'u' 10100 xor 'u' 10100 = 00000 ( 0) 'a'
'i' 01000 xor 'e' 00100 = 01100 (12) 'm'
'p' 01111 xor 's' 10010 = 11101 (29) (no es letra)
'o' 01110 xor 'e' 00100 = 01010 (10) 'k'
'd' 00011 xor 'c' 00010 = 00001 ( 1) 'b'
'i' 01000 xor 'r' 10001 = 11001 (25) 'z'
'e' 00100 xor 'e' 00100 = 00000 ( 0) 'a'
'z' 11001 xor 't' 10011 = 01010 (10) 'k'

--- Cálculo del descifrado (fórmula M = C xor K) ---
01101 xor 'k' 01010 = 00111 ( 7) -> 'h'
01010 xor 'e' 00100 = 01110 (14) -> 'o'
10011 xor 'y' 11000 = 01011 (11) -> 'l'
10101 xor 'v' 10101 = 00000 ( 0) -> 'a'
00100 xor 'a' 00000 = 00100 ( 4) -> 'e'
11011 xor 'l' 01011 = 10000 (16) -> 'q'
00000 xor 'u' 10100 = 10100 (20) -> 'u'
01100 xor 'e' 00100 = 01000 ( 8) -> 'i'
11101 xor 's' 10010 = 01111 (15) -> 'p'
01010 xor 'e' 00100 = 01110 (14) -> 'o'
00001 xor 'c' 00010 = 00011 ( 3) -> 'd'
11001 xor 'r' 10001 = 01000 ( 8) -> 'i'
00000 xor 'e' 00100 = 00100 ( 4) -> 'e'
01010 xor 't' 10011 = 11001 (25) -> 'z'

------------ Resultados ------------

--> Clave utilizada:          keyvaluesecret
--> Mensaje original:         holaequipodiez
--> Criptograma (bits):       01101 01010 10011 10101 00100 11011 00000 01100 11101 01010 00001 11001 00000 01010
--> Criptograma (decimal):    13 10 19 21 4 27 0 12 29 10 1 25 0 10
--> Mensaje descifrado:       holaequipodiez

--------------------------------------
```

Figura 1: Prueba1

El criptograma en decimal es `13 10 19 21 4 27 0 12 29 10 1 25 0 10`. Al descifrar se recupera `holaequipodiez` sin ningún error.

3.2. Prueba 2: otro mensaje y otra clave

Se cifró “mensaje secreto” (14 letras) con la clave `llavesecretaxx`, que tiene justo 14 letras.

```
  Algoritmo de Cifrado Vernam

> Ingrese el mensaje a cifrar: mensaje secreto
> Ingrese la clave de al menos 14 letras (Enter para generarla aleatoria): llavesecretaxx

--- Cálculo del cifrado (fórmula C = M xor K) ---
'm' 01100 xor 'l' 01011 = 00111 ( 7) 'h'
'e' 00100 xor 'l' 01011 = 01111 (15) 'p'
'n' 01101 xor 'a' 00000 = 01101 (13) 'n'
's' 10010 xor 'v' 10101 = 00111 ( 7) 'h'
'a' 00000 xor 'e' 00100 = 00100 ( 4) 'e'
'j' 01001 xor 's' 10010 = 11011 (27) (no es letra)
'e' 00100 xor 'e' 00100 = 00000 ( 0) 'a'
's' 10010 xor 'c' 00010 = 10000 (16) 'q'
'e' 00100 xor 'r' 10001 = 10101 (21) 'v'
'c' 00010 xor 'e' 00100 = 00110 ( 6) 'g'
'r' 10001 xor 't' 10011 = 00010 ( 2) 'c'
'e' 00100 xor 'a' 00000 = 00100 ( 4) 'e'
't' 10011 xor 'x' 10111 = 00100 ( 4) 'e'
'o' 01110 xor 'x' 10111 = 11001 (25) 'z'

--- Cálculo del descifrado (fórmula M = C xor K) ---
00111 xor 'l' 01011 = 01100 (12) -> 'm'
01111 xor 'l' 01011 = 00100 ( 4) -> 'e'
01101 xor 'a' 00000 = 01101 (13) -> 'n'
00111 xor 'v' 10101 = 10010 (18) -> 's'
00100 xor 'e' 00100 = 00000 ( 0) -> 'a'
11011 xor 's' 10010 = 01001 ( 9) -> 'j'
00000 xor 'e' 00100 = 00100 ( 4) -> 'e'
10000 xor 'c' 00010 = 10010 (18) -> 's'
10101 xor 'r' 10001 = 00100 ( 4) -> 'e'
00110 xor 'e' 00100 = 00010 ( 2) -> 'c'
00010 xor 't' 10011 = 10001 (17) -> 'r'
00100 xor 'a' 00000 = 00100 ( 4) -> 'e'
00100 xor 'x' 10111 = 10011 (19) -> 't'
11001 xor 'x' 10111 = 01110 (14) -> 'o'

------------ Resultados ------------

--> Clave utilizada:          llavesecretaxx
--> Mensaje original:         mensajesecreto
--> Criptograma (bits):       00111 01111 01101 00111 00100 11011 00000 10000 10101 00110 00010 00100 00100 11001
--> Criptograma (decimal):    7 15 13 7 4 27 0 16 21 6 2 4 4 25
--> Mensaje descifrado:       mensajesecreto

-------------------------------------
```

Figura 2: Prueba 2

3.3. Nota sobre la seguridad

Estas pruebas usan claves escritas a mano, solo para poder verificar el resultado. Para que Vernam sea seguro de verdad, la clave debe ser realmente aleatoria, no se debe reutilizar nunca y solo la deben conocer quien cifra y quien descifra. Si se usa la misma clave dos veces, un atacante puede hacer XOR de los dos criptogramas y obtener información de los mensajes. Además, el módulo `random` de Python no es de calidad criptográfica; para un uso real conviene el módulo `secrets`.

4. Repositorio

El código de esta práctica 06 se puede consultar en el siguiente repositorio:

https://github.com/percivalespi/Criptografia/blob/main/practica6/Algoritmo_Cifrado_Vernam.py

5. Conclusión

En esta práctica aprendimos cómo funciona la operación XOR en el cifrado de Vernam y por qué se utiliza el mismo proceso para cifrar y descifrar. Lo más interesante fue ver que, al trabajar con 5 bits, el resultado puede tener valores de 26 a 31, que no corresponden a ninguna letra del alfabeto. Por eso, el mensaje cifrado se guarda como una cadena de bits y no como texto. También entendimos que la seguridad de este método depende principalmente de la clave, ya que debe ser aleatoria, tener el mismo largo que el mensaje y utilizarse una sola vez. Si no se cumplen estas condiciones, el cifrado deja de ser seguro.